UNIVERSIDADE FEDERAL DO AMAZONAS
PRÓ-REITORIA DE PESQUISA E PÓS-GRADUAÇÃO
DEPARTAMENTO DE PESQUISA
PROGRAMA INSTITUCIONAL DE BOLSAS DE INICIAÇÃO CIENTÍFICA - PIBIC
PROGRAMA DE APOIO À INICIAÇÃO CIENTÍFICA - PAIC

RELATÓRIO FINAL

EDIÇÃO: PIBIC/PAIC 2020/2021	
RECURSOS HUMANOS	
Nome do(a) orientador(a): Raimundo da Silva Barreto	
Nome do(a) aluno(a) Lucas do Nascimento Silva	**Bolsa:** (X) CNPQ ( ) UFAM ( ) FAPEAM ( ) VOLUNTÁRIO
IDENTIFICAÇÃO DO PROJETO	
Título: Um Ambiente de Programação para o Microcontrolador ESP32 Usando Computação Móvel e Agente de IA do ChatGPT	**Código do Projeto:** PIB-E/0011/2024
Área de Conhecimento: (X) Exatas e da Terra ( ) Agrárias ( ) Biológicas ( ) Sociais Aplicadas ( ) Engenharias ( ) Saúde ( ) Ciências Humanas ( ) Linguística, Letras e Artes ( ) Multidisciplinar	
COMITÊ DE ÉTICA EM PESQUISA COM HUMANOS (CEP) OU ANIMAIS (CEUA)	
( ) Aprovado - Número do protocolo: _____ (X) Não se aplica Caso o projeto ainda não esteja aprovado, justifique:	

* O Relatório deve ser apresentado abaixo deste formulário em no máximo 20 páginas
* O Relatório deverá estar de acordo com as normas atualizadas da ABNT para trabalhos acadêmicos.

RESUMO

Este projeto teve como foco o desenvolvimento de um ambiente de programação automatizado para o microcontrolador ESP32, com apoio de técnicas de Inteligência Artificial e de um backend em Django. A proposta buscou reduzir a barreira técnica enfrentada por iniciantes na área de sistemas embarcados, possibilitando a geração de código em C/C++ a partir de instruções em linguagem natural.

O sistema foi estruturado de forma a iniciar uma conversa com o usuário, conduzida por um agente baseado no modelo ChatGPT, adaptado para o domínio específico do ESP32. Embora a integração direta com o agente personalizado da OpenAI não fosse possível por limitações da

API, solucionou-se essa restrição com a incorporação do prompt do agente no backend, garantindo que o modelo reproduzisse seu comportamento.

Além da integração com a API da OpenAI, foi criado um mecanismo de fallback local que assegura a resiliência do sistema, gerando exemplos básicos quando não há chave válida ou créditos disponíveis. A arquitetura em Django permitiu a criação de uma interface web simples, responsiva e escalável, incluindo rotas REST, banco de dados para registro de interações e preparação para envio OTA (Over-The-Air).

Os resultados demonstraram a viabilidade do ambiente proposto, validando sua aplicabilidade em contextos educacionais e de prototipagem rápida. O projeto representa uma contribuição para a democratização do uso do ESP32 e para a integração da Inteligência Artificial no processo de ensino-aprendizagem de programação embarcada.

INTRODUÇÃO

A área de Internet das Coisas (IoT) vem apresentando crescimento expressivo nos últimos anos, com aplicações em diversos setores, como automação residencial, cidades inteligentes, saúde, transporte e indústria 4.0. Dentro desse cenário, o ESP32 destaca-se como um microcontrolador de baixo custo e alto desempenho, amplamente utilizado pela comunidade acadêmica e maker. Seu conjunto de recursos — conectividade Wi-Fi e Bluetooth, múltiplos GPIOs, suporte a diversos sensores e módulos externos — torna-o uma escolha popular para prototipagem e pesquisa.

Apesar dessas vantagens, a programação do ESP32 ainda constitui um desafio para iniciantes. O processo requer domínio da linguagem C/C++, familiaridade com a IDE Arduino e compreensão da arquitetura de hardware, o que pode desmotivar estudantes e dificultar a adoção em disciplinas introdutórias. Surge, assim, a necessidade de ferramentas que abstraiam a complexidade técnica e tornem a experiência de desenvolvimento mais acessível.

Com a popularização de modelos de linguagem como o ChatGPT, abre-se a possibilidade de integrar agentes conversacionais ao processo de desenvolvimento, permitindo que um usuário

descreva em linguagem natural o que deseja que o microcontrolador execute, e o sistema traduza essa descrição em código executável. No entanto, a API da OpenAI não disponibiliza acesso direto a “agentes customizados” criados pelo usuário, restringindo a integração a prompts de sistema.

Diante desse cenário, este projeto teve como objeto de estudo a implementação de um ambiente web em Django que incorpora o prompt de um agente especializado em ESP32 diretamente no backend. A solução possibilita que o modelo atue de forma guiada, conduzindo o usuário por meio de perguntas estruturadas (objetivo do projeto, sensores, atuadores, pinos utilizados, conectividade, armazenamento e visualização de dados) e, ao final, gere automaticamente um codigo para o ESP32.

Além disso, o sistema foi planejado para contemplar a funcionalidade de envio OTA (Over-The-Air), permitindo que, futuramente, o código gerado possa ser transferido diretamente para o microcontrolador pela rede, sem necessidade de conexão física via cabo USB. Essa característica amplia a usabilidade da solução, tornando-a mais prática e alinhada com cenários reais de IoT, nos quais a atualização remota de dispositivos é um requisito essencial.

A justificativa para o estudo repousa na busca por ferramentas que democratizem a programação embarcada, favoreçam a aprendizagem prática em ambientes acadêmicos e estimulem a prototipagem rápida em laboratórios de inovação.

OBJETIVOS

Objetivo Geral

Desenvolver e validar um ambiente web que atue como uma plataforma de apoio à programação do microcontrolador ESP32, capaz de interpretar instruções fornecidas em linguagem natural e convertê-las em código executável em C/C++ compatível com a IDE Arduino. O ambiente deve ser projetado para guiar o usuário passo a passo por meio de um fluxo conversacional, conduzido por um agente de Inteligência Artificial configurado a partir de um prompt especializado.

Além de gerar o código de forma automatizada, o sistema deve assegurar resiliência operacional por meio de um mecanismo de fallback local, garantindo que o usuário sempre receba um resultado, mesmo em situações de indisponibilidade ou restrição da API da OpenAI. Outro ponto central é a preparação para envio OTA (Over-The-Air), que viabilizará a atualização remota do microcontrolador sem necessidade de conexão física, alinhando a solução às demandas reais de projetos em IoT.

O objetivo geral, portanto, além do desenvolvimento de um protótipo funcional, é também promover acessibilidade e democratização do aprendizado em programação embarcada, criando um recurso útil tanto para estudantes iniciantes quanto para pesquisadores e desenvolvedores que desejam acelerar o processo de prototipagem de dispositivos inteligentes.

Objetivos Específicos

1. Implementar no backend Django um agente especializado em geração de código para ESP32, a partir de prompts estruturados.

2. Criar uma interface web em Django (HTML, CSS, JS) que permita ao usuário interagir em linguagem natural com o agente.

3. Estruturar um fluxo conversacional guiado (entrevista passo a passo) para coletar informações do usuário e gerar o código correspondente.

4. Integrar a geração de código ao backend, incluindo fallback local para cenários sem API Key.

5. Registrar interações e resultados em banco de dados.

6. Preparar a infraestrutura para integração futura com envio OTA.

METODOLOGIA

A metodologia adotada foi dividida em etapas sequenciais, abrangendo planejamento, desenvolvimento, integração, testes e análise dos resultados obtidos.

1. Planejamento e definição do agente

Foi estruturado um roteiro de entrevista composto por perguntas sequenciais que cobrem desde o objetivo do projeto até detalhes técnicos como sensores, atuadores e conectividade.

Esse roteiro foi incorporado como prompt de sistema no backend, de forma que o modelo de IA se comporte como um agente especializado em ESP32.

Optou-se por não depender do agente customizado criado na interface do ChatGPT, mas sim replicar seu comportamento via prompt, garantindo maior controle e portabilidade.

Essa etapa foi fundamental para garantir que o agente conduzisse o diálogo de forma estruturada, realizando perguntas sucessivas até coletar todas as informações necessárias. O fluxo foi pensado de modo a tolerar respostas fora de ordem, prosseguindo apenas com as questões ainda não respondidas, o que reduz ambiguidades e torna o sistema acessível até para iniciantes.

2. Desenvolvimento do backend em Django

Criou-se o projeto ESPAssist e o aplicativo ESPAssists, com as seguintes funcionalidades:

- **Endpoints REST**: `/api/agent/next` e `/api/agent/reset` para gerenciar o fluxo conversacional, `/api/generate` para geração direta de código e `/api/ota` como stub para envio OTA.

- **Serviço de geração (`generator.py`)**: integração com a API da OpenAI para geração de código e implementação do fallback local.

- **Banco de dados SQLite**: persistência de prompts, respostas, códigos e feedbacks.

O uso do Django como framework foi motivado por sua modularidade e integração nativa com banco de dados, além da facilidade de manutenção e escalabilidade. Essa estrutura assegura que o backend seja robusto, organizado e preparado para futuras expansões.

3. Interface web (frontend)

- A interface foi construída em **HTML, CSS e JavaScript**, integrada via templates do Django.
- Foi implementado um fluxo **interativo**, em que o agente apresenta a primeira pergunta logo ao carregar a página. O usuário responde passo a passo até completar o briefing; em seguida, o sistema gera e exibe o código, com botões de **copiar** e **enviar OTA**.
- O **design responsivo** permite o uso em diferentes dispositivos, incluindo computadores e smartphones, favorecendo a acessibilidade e a praticidade.

4. Integração com a OpenAI e fallback local

O backend utiliza o modelo gpt-4o-mini por ser mais rápido e econômico, mantendo a possibilidade de uso do gpt-4o.

Implementou-se também o fallback local, responsável por gerar exemplos básicos (como piscar um LED ou leitura analógica) em caso de falha na chamada à API.

Essa decisão garantiu que o sistema fosse resiliente, permitindo testes mesmo sem chave válida ou sem crédito na OpenAI. Além disso, o fallback também tem caráter pedagógico, já que fornece exemplos simples que podem ser usados em contextos de ensino.

5. Testes e validação

- **Testes unitários**: feitos via `curl`, garantindo que cada endpoint retornasse JSON válido.

- **Testes de integração**: foram conduzidas interações completas pelo navegador, verificando a correta comunicação entre frontend e backend, assim como a exibição do código gerado na interface.

- **Cenários de falha**: como a remoção da chave da OpenAI ou a extrapolação do limite de uso, para validar o funcionamento do fallback local. Isso comprovou que o sistema é capaz de continuar operando mesmo diante de restrições externas.

RESULTADOS/DISCUSSÃO

O projeto resultou em um protótipo funcional que demonstra a viabilidade do uso de IA para auxiliar na programação do ESP32.

Resultados alcançados

- Backend em Django com endpoints REST para gerenciamento da entrevista e geração de código.

- Interface web funcional, permitindo ao usuário interagir em linguagem natural com o agente.

- Geração automática de código Arduino/C++ a partir das respostas fornecidas pelo usuário.

- Implementação de fallback local, assegurando resiliência do sistema.

- Registro de interações no banco de dados, possibilitando auditoria e futuras análises.

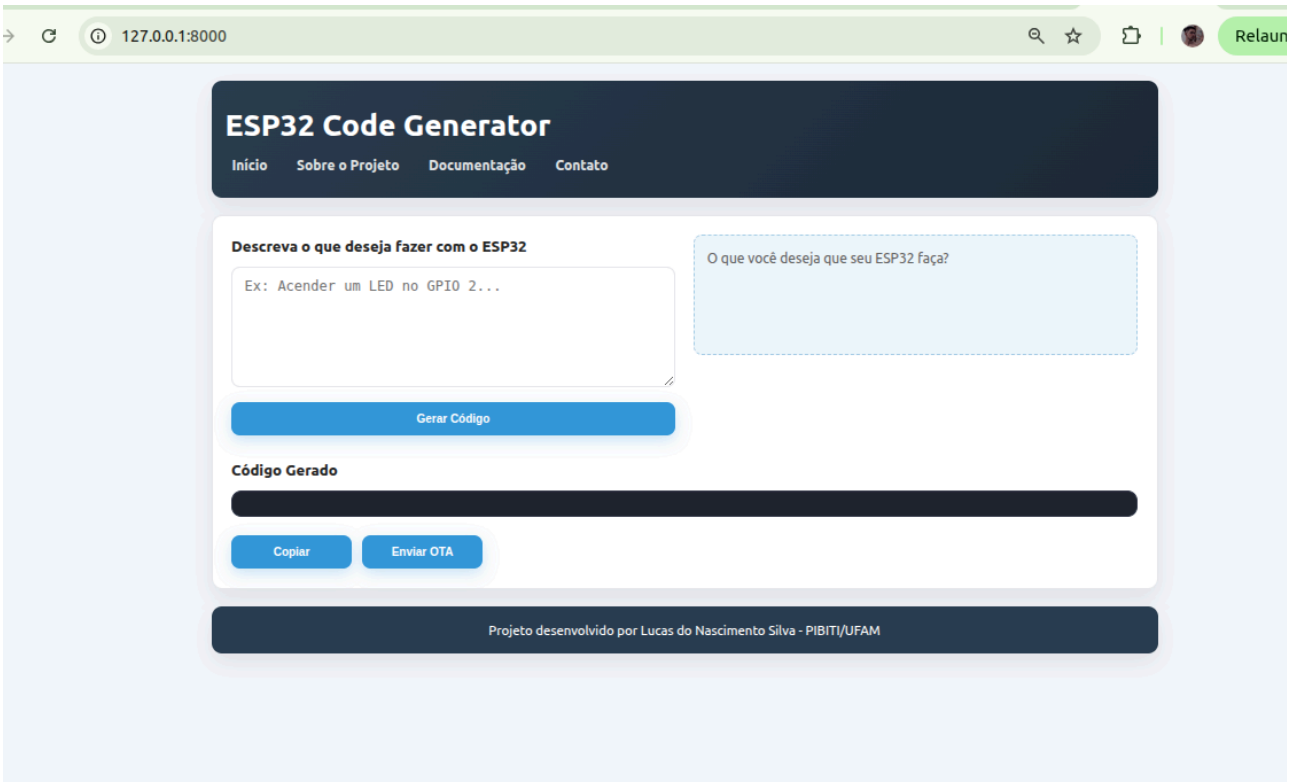

Figura 1: Pagina Inicial da plataforma web

Discussão

A impossibilidade de integrar diretamente o agente customizado da OpenAI não comprometeu os objetivos, pois o uso de prompts no backend se mostrou uma solução eficiente. Essa abordagem garante maior flexibilidade, permitindo ajustes contínuos no comportamento do agente sem depender de limitações externas.

O fallback local foi uma decisão estratégica, pois além de garantir o funcionamento sem chave ou crédito, também serve como recurso pedagógico, fornecendo exemplos simples que podem ser utilizados em aulas introdutórias.

A escolha do Django como framework se mostrou adequada pela modularidade, integração com banco de dados e facilidade de manutenção, preparando o sistema para futuras expansões (como integração móvel e OTA real).

Entre as limitações, se destacam:

- Dependência de créditos na OpenAI para uso pleno da IA;
- Funcionalidade OTA ainda não implementada na prática (apenas como stub);
- Necessidade de expandir a base de exemplos e testes para cobrir mais sensores e atuadores.

CONCLUSÃO

O trabalho alcançou os objetivos propostos, construindo um ambiente web utilizando o framework Django, que permite a criação rápida e eficiente de aplicações web robustas, escaláveis e seguras, fornecendo um conjunto abrangente de ferramentas reutilizáveis e "baterias incluídas" para tarefas comuns, como autenticação e interação com bancos de dados. A plataforma web criada permite a geração automática de código para ESP32 a partir de instruções em linguagem natural. A integração com a OpenAI, mesmo sem suporte direto a agentes customizados, foi realizado pela injeção do prompt no backend, reproduzindo o comportamento esperado do agente.

A interface desenvolvida mostrou-se adequada tanto para fins educacionais quanto para prototipagem rápida, tornando o processo de programação mais acessível a iniciantes. O fallback local acrescentou robustez ao sistema, assegurando funcionamento mínimo em cenários adversos.

O projeto deixa como perspectivas futuras:

- integração real com OTA para envio direto do código ao ESP32,
- criação de aplicativo móvel para ampliar acessibilidade,
- expansão da biblioteca de exemplos e suporte a múltiplos sensores,
- avaliação quantitativa da qualidade do código gerado em contextos reais de uso.

Assim, este trabalho reforça o potencial da Inteligência Artificial aplicada à programação embarcada, oferecendo uma alternativa inovadora e acessível que pode transformar tanto o ensino quanto a prática de IoT.

REFERÊNCIAS

Hercog D, Lerher T, Truntic M, Težak O. Design and Implementation of ESP32-Based IoT Devices. *Sensors*. 2023;23:6739. Disponível em: https://doi.org/10.3390/s23156739

Pasic R, Kuzmanov I, Atanasovski K. ESP-NOW Communication Protocol with ESP32. *Journal of Universal Excellence*. 2021;6(1):53-60. ISSN 2232-5204.

Kurniawan A. *Internet of Things Projects with ESP32*. Birmingham: Packt Publishing; 2019. ISBN: 978-1789956870.

Pravalika V, Prasad CR. Internet of Things Based Home Monitoring and Device Control Using ESP32. *International Journal of Recent Technology and Engineering (IJRTE)*. 2019;8(1S4). ISSN: 2277-3878.

Opara E, Mfon-Ette T, Adalikwu A, Aduke TC. ChatGPT for Teaching, Learning and Research: Prospects and Challenges. *Global Academic Journal of Humanities and Social Sciences*. 2023;5(2):33-40. Disponível em: https://ssrn.com/abstract=4375470

Wu T, He S, Liu J, Sun S, Liu K, Han QL, Tang Y. A Brief Overview of ChatGPT: The History, Status Quo and Potential Future Development. *IEEE/CAA Journal of Automatica Sinica*. 2023;10(5):1122-36. DOI: 10.1109/JAS.2023.123618

San Pietro CB di, Frau F, Mangiaterra V, Bambini V. The Pragmatic Profile of ChatGPT: Assessing the Communicative Skills of a Conversational Agent. *Sistemi intelligenti: Rivista quadrimestrale di scienze cognitive e di intelligenza artificiale*. 2023;2:379-400. DOI: 10.1422/108136